Parallelizing LLaVA

ML710 course project · MBZUAI

Rassul Magauin · Omar Ahmed · Youssef Ghallab

Spring 2026

Setup

- MBZUAI HPC, ws-ia partition: 2 nodes × 1 RTX 5000 Ada (32 GB), Ethernet ~1 Gbps.
  - No NVLink, no InfiniBand. Cross-node comm goes over a slow link.
  - The gpu partition has a 1-GPU-per-user QOS, so multi-GPU = two salloc sessions + torchrun.
- Software: PyTorch 2.1.2, DeepSpeed 0.12.6, Transformers 4.37.2, peft 0.6.0.
- Team of three, each owning at least one non-trivial parallelism strategy:
  - Pipeline parallelism — Youssef Ghallab
  - DeepSpeed ZeRO-1, ZeRO-2, and ZeRO-3 — Omar Ahmed
  - PyTorch FSDP and Tensor Parallelism — Rassul Magauin

The model: LLaVA

- Liu et al., "Visual Instruction Tuning" (NeurIPS 2023). Multimodal: image + text → text.
- About 7B parameters total, in three pieces:
  - CLIP ViT-L/14-336 vision encoder — frozen.
  - 2-layer MLP projection — small, trainable.
  - Vicuna-7B language model — frozen in Stage 1, fine-tuned in Stage 2.
- Two-stage training:
  - Stage 1 trains only the projection (~80 MB) on 558K image-caption pairs.
  - Stage 2 fine-tunes the projection plus the LLM on 665K instructions.
- We use Stage 2 because it has meaningful gradient communication. Stage 1's AllReduce is too small to matter.

Stage 2 — what we trained

- Starting checkpoint: liuhaotian/llava-v1.5-7b. The CLIP and MLP were already trained in Stage 1.
- Dataset: 10K random samples from llava_instruct_150k.json with COCO 2014 train images. One epoch keeps each run under an hour.
- Training target: LoRA adapters (r=128, α=256) on the LLM, plus the full MLP projection.
  - ≈4.6% of total parameters — about 340M of 7.4B.
- AdamW, bf16, gradient checkpointing, cosine schedule.
- Effective batch sizes vary so each method actually fits in memory:
  - DDP, ZeRO-1, ZeRO-2: 32 (per_device=16, world=2)
  - Pipeline: 16 (per_device=1, accum=16, world=2)
  - ZeRO-3, FSDP, TP: 8 (per_device=1, accum=4, world=2)
  - TP trains the projection only — no LoRA, since LoRA on TP-sharded base linears is fragile in PyTorch 2.1.

What we compared

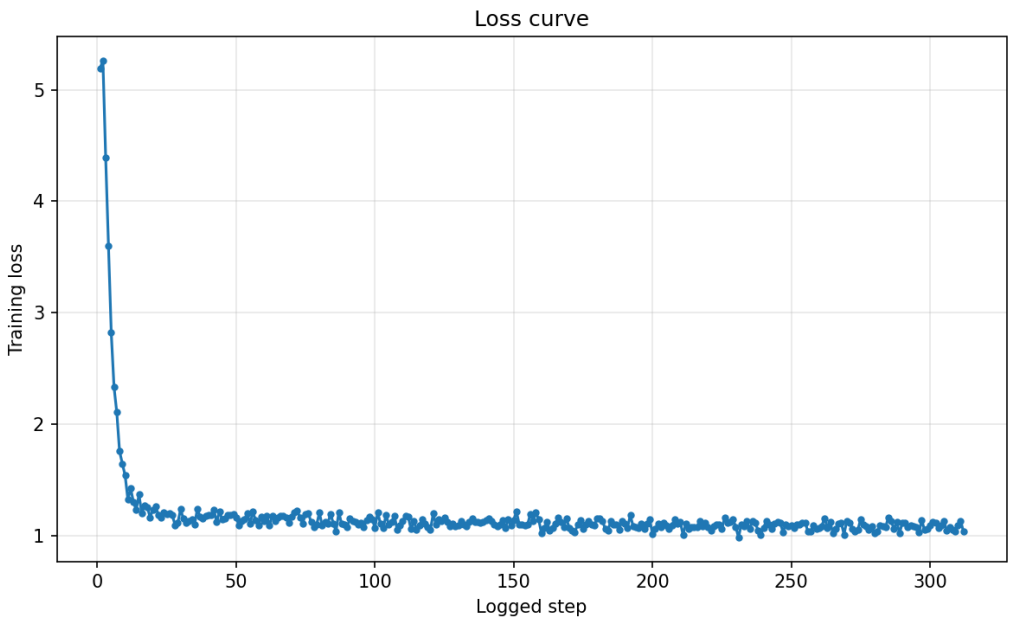
- Single-GPU LoRA — reference for what no parallelism looks like.
- DDP (DeepSpeed ZeRO-0) — naïve baseline; the model is replicated, gradients are AllReduce'd.
- Pipeline parallelism — the model is split across the two nodes layer-wise. Owned by Ghallab.
- ZeRO-1 — DDP with sharded optimizer states. Owned by Omar.
- ZeRO-2 — same plus sharded gradients. Owned by Omar.
- ZeRO-3 — same plus sharded parameters. Owned by Omar.
- PyTorch FSDP — PyTorch-native equivalent of ZeRO-3. Owned by Rassul.
- Tensor Parallel — splits each layer's matmuls across the two nodes. Owned by Rassul.
- Every multi-node run uses the same two ws-ia nodes, same Ethernet link.

Baseline 1: single GPU

- One RTX 5000 Ada, batch size 1, 10000 steps for one epoch over the 10K subset.
- Wall time: 96 minutes.
- Throughput: 1.74 samples/sec.
- Peak GPU memory: 25.7 GiB out of 32. Fits comfortably.
- Average GPU utilization: 11%.
  - CPU-bound on data preprocessing — dataloader_num_workers=0, single-threaded image loading.
  - The GPU spends most of its time waiting for the next batch.
- Final loss: 1.32 (avg 1.10 across the run).
- Reference for what 'no parallelism' looks like before we add anything.

Baseline 2: DDP across two nodes

- PyTorch DistributedDataParallel via torchrun --nnodes=2.
  - DeepSpeed ZeRO-0 — no sharding, each rank holds the whole model.
- Per-device batch 16, effective batch 32, 313 steps for one epoch.
- Wall time: 74.8 min — about 1.3× faster than single GPU.
- Throughput: 2.22 samples/sec.
- Peak GPU memory: 30.7 GiB.
- GPU utilization: 70.7%.
  - Visible AllReduce stalls when both nodes wait on cross-node Ethernet.
- Final loss: 1.04. This is the reference everything else is compared against.



Pipeline Parallelism

How we built it (Ghallab)

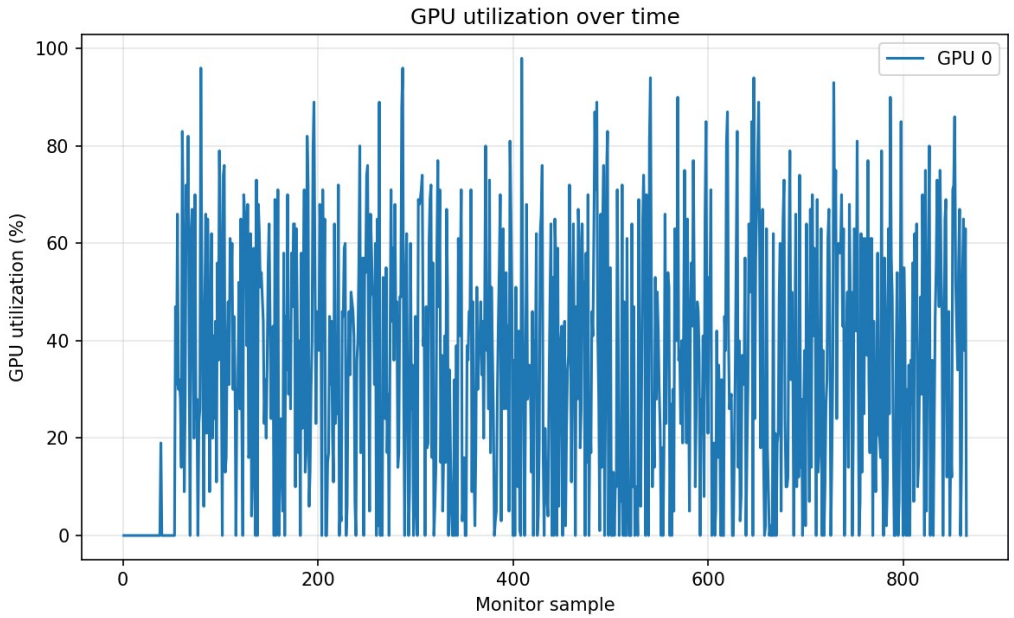
- Manual two-stage pipeline. We didn't use torchgpipe or DeepSpeed-Pipeline.
  - Custom training loop: LLaVA/llava/train/train_pipe_dist.py (~520 lines).
  - Gloo backend for cross-node send/recv. NCCL had ordering issues with our mixed-dtype activations.
- The model is cut at decoder layer k. There are 32 decoder layers in Vicuna-7B.
  - Rank 0 holds the vision tower (frozen), the MLP projection, and decoder layers 0..k-1.
  - Rank 1 holds layers k..31, the final norm, the LM head, and computes loss.
  - Activations flow forward, gradients flow back, by hand.
- Gradient accumulation 16, microbatch 1 → effective batch 16. LoRA is added on each rank's local layers.

Finding a split that fits

- Picking the layer index k matters more than expected.
- k = 16 (50/50 by layer count): rank 0 OOMs in the first forward.
  - Vision tower + image features sit on rank 0. With layers 0..15 it hits 31.4 GiB — over the 32 GiB card.
- k = 8 (push work to rank 1): rank 1 OOMs.
  - LM head + final norm + layers 8..31 hit 31.0 GiB.
- k = 12 with microbatch 1 + grad accum 16: both ranks fit.
  - Rank 1 ≈ 31.4 GiB, rank 0 ≈ 28 GiB. Run completes 625/625 steps.
- Lesson: layer count is a poor proxy for memory cost. The vision tower and LM head are heavier than they look.

Pipeline — results

- Wall time: 72.1 min. Almost identical to DDP.
- Throughput: 2.31 samples/sec — 1.04× DDP. Barely faster.
- Effective batch 16, half of DDP, so fewer steps per epoch.
- Peak GPU memory: 31.4 GiB on the heavier rank.
- GPU utilization is uneven: rank 0 at 34%, rank 1 at 56%.
  - Whichever rank finishes its layers first sits idle until the other catches up.
  - Gloo is much slower than NCCL, so the comm gap is wider.
- Final loss: 1.57 — higher than DDP because of the smaller effective batch.
- Pipeline made the model fit. It did not make it faster.



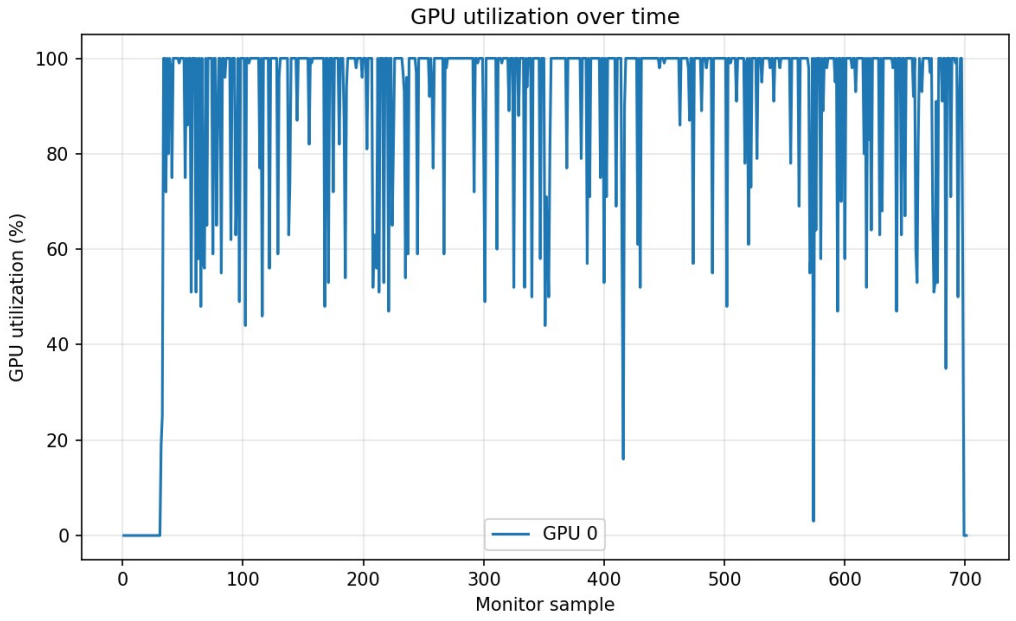
DeepSpeed ZeRO-1 / ZeRO-2

ZeRO-1 vs ZeRO-2 (Omar)

- Both are data parallelism with sharded state. The difference is what gets sharded.
- ZeRO-1:
  - Optimizer states sharded. Gradients still AllReduce'd (like DDP), parameters replicated.
  - After the optimizer step, AllGather to put updated parameters back on every rank.
- ZeRO-2 adds gradient sharding via ReduceScatter:
  - Gradients land on their owner-rank shards directly, no AllReduce.
  - Each rank takes one optimizer step on its shard, then AllGather params back.
  - Comm volume per step is roughly 1.5× the model size — same as ZeRO-1, different schedule.
- On a LoRA workload, both win mostly from DeepSpeed's bucketed async overlap, not from the sharding itself.
  - ZeRO-1 throughput is between DDP and ZeRO-2; ZeRO-2 is fastest because ReduceScatter overlaps better with backward.

ZeRO-1 / ZeRO-2 — results

- Both at effective batch 32, full epoch (313 steps).
- ZeRO-1: 65 min, 2.56 sps, 28.8 GiB, 81% util, loss 1.14 — 1.15× DDP.
- ZeRO-2: 58 min, 2.85 sps, 27.6 GiB, 89% util, loss 1.11 — 1.28× DDP.
  - The only method that beats plain DDP outright.
- Why ZeRO-2 edges out ZeRO-1: ReduceScatter overlaps with the backward pass, AllReduce doesn't as cleanly.
- Why the speedup over DDP isn't 2×: LoRA's optimizer state is tiny — most of the win is overlap, not sharding.



DeepSpeed ZeRO-3

ZeRO-3: shard everything (Omar)

- Same data-parallel semantics as ZeRO-2. Each rank still sees a different microbatch.
- The difference: parameters are also sharded across the two ranks.
- Per-step comm now happens at every layer, twice:
  - Forward pass: AllGather parameters for layer N → compute → drop them.
  - Backward pass: AllGather them again → compute gradients → drop again.
  - ReduceScatter the gradients at the end of backward, then run the optimizer on each shard.
- On 2 ranks, ~7 GB of parameters live on each. But every layer briefly re-materializes its full ~430 MB on every rank.
- Back-of-envelope per microbatch:
  - AllGather other rank's 7 GB shard, fwd + bwd → ~14 GB cross-node per microbatch.
  - × 4 microbatches per optimizer step + ReduceScatter at end ≈ ~63 GB on the wire per step.
  - Over 1 Gbps Ethernet that's minutes of pure comm — DeepSpeed bucketed-async overlap (50 MB) is the only thing keeping it tractable.
- We used configs/zero3_bounded_async.json to cap allgather/reduce buckets.

ZeRO-3 — results

- Hit the SLURM time limit before finishing. Reached 292 of 1250 steps in 3 h 52 min.
  - Each step took about 48 seconds. Projected full run: 16.5 hours.
- Throughput: 0.17 samples/sec — about 13× slower than DDP.
- Peak GPU memory: 22 GiB — actual sharding showing up.
  - But memory was never the bottleneck on this card. Communication was.
- GPU utilization: 99%. This is misleading.
  - DeepSpeed pipelines comm with compute, so 'utilization' includes time spent waiting on AllGather.
  - High util doesn't mean fast — it means the engine kept the GPU busy on something.
- Last logged loss: 1.31. Convergence was fine — the run was just too slow to finish.
- Diagnosis: per-layer AllGather over 1 Gbps Ethernet is the wall.

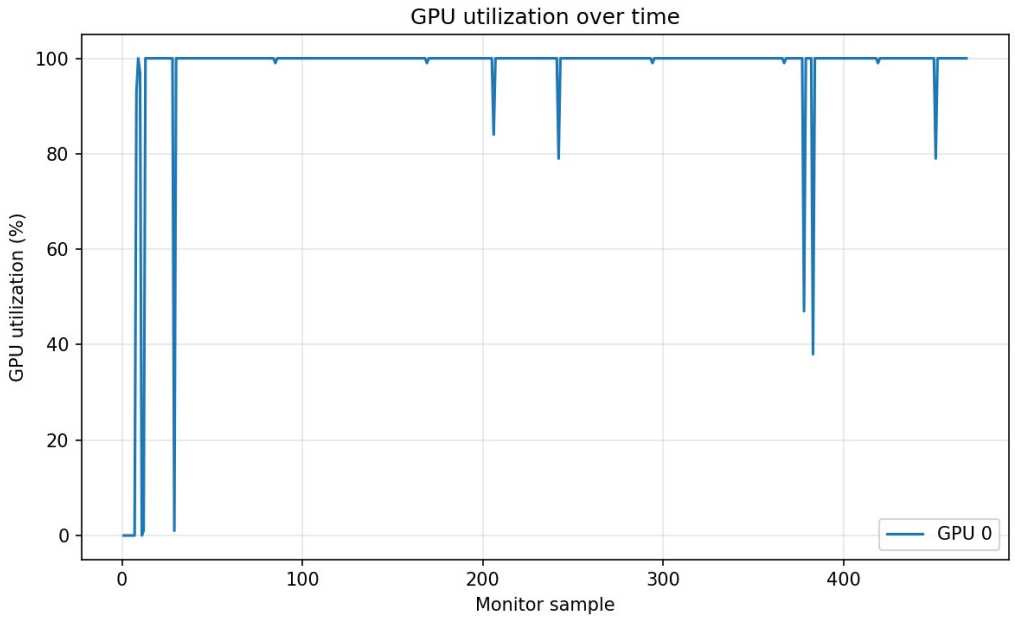
PyTorch FSDP

FSDP: PyTorch's take (Rassul)

- PyTorch-native equivalent of ZeRO-3. Same algorithm, different implementation.
- Design choices that differ from DeepSpeed:
  - Each transformer block is wrapped as one FSDP unit (a contiguous FlatParameter).
  - AllGather happens at the unit boundary, not per-tensor.
  - Mixed precision is set explicitly via a policy (we used bf16).
- Our config: full_shard auto_wrap on LlamaDecoderLayer, use_orig_params=True (required for LoRA), cpu_ram_efficient_loading=true.
- Two LLaVA-specific bugs we had to patch to get this running:
  - self.vision_tower[0] crashes when loading from a vendored checkpoint where vision_tower isn't a list.
  - LoRA adapters are created in fp32 after the base model is cast to bf16. FSDP requires uniform dtype, so we cast LoRA to bf16 too.

FSDP — results

- Ran to step 292 — matched ZeRO-3's stopping point — at 84 s/step.
- Wall time: 6.8 h. Throughput: 0.095 sps — ~22× slower than DDP, ~1.8× slower than ZeRO-3.
- Peak GPU memory: 31.6 GiB (right at the card's limit).
- GPU utilization: 97.5%. Final loss: 1.34 — matches ZeRO-3 within noise (same algorithm).
- Why slower than ZeRO-3 on the same hardware?
  - DeepSpeed's bounded async (50 MB buckets) overlaps comm with compute aggressively.
  - FSDP's defaults wait for each block's AllGather to finish before computing.
  - Same algorithm, different async behavior — the difference is real on a slow link.



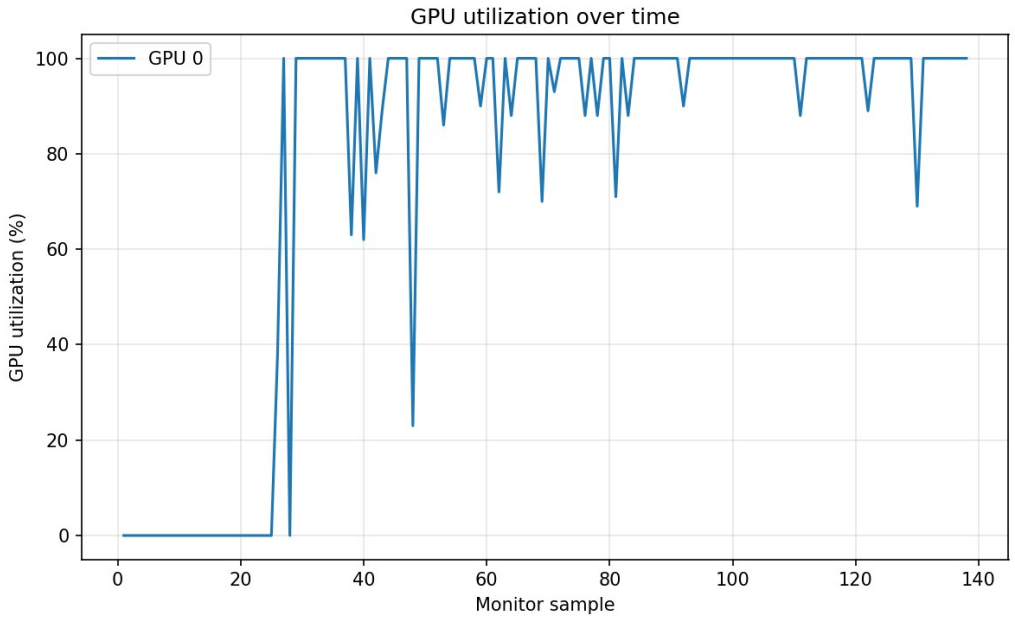
Tensor Parallelism

Tensor Parallel: split the matmuls (Rassul)

- Megatron-style intra-layer model parallelism. Each LlamaDecoderLayer's linear sublayers are sharded column-wise or row-wise across the two ranks.
- Sharding plan, applied per layer via torch.distributed.tensor.parallel.parallelize_module:
  - q_proj, k_proj, v_proj, gate_proj, up_proj — Colwise (output dim split).
  - o_proj, down_proj — Rowwise (input dim split, AllReduce on output).
  - After parallelize, num_heads / num_key_value_heads / hidden_size are halved per rank.
- Per-step communication: ~6 cross-node AllReduces per layer × 32 layers × 2 (fwd+bwd) ≈ 384 collectives per microbatch, × 4 grad-accum.
- We trained the MLP projection only — LoRA on TP-sharded base linears is fragile in PyTorch 2.1.2 (DTensor + LoRA's lora_A/B reshape don't compose cleanly).
- Custom 2-rank torchrun launcher: LLaVA/llava/train/train_tp_dist.py + scripts/run_stage2_tp.sh.

TP — results

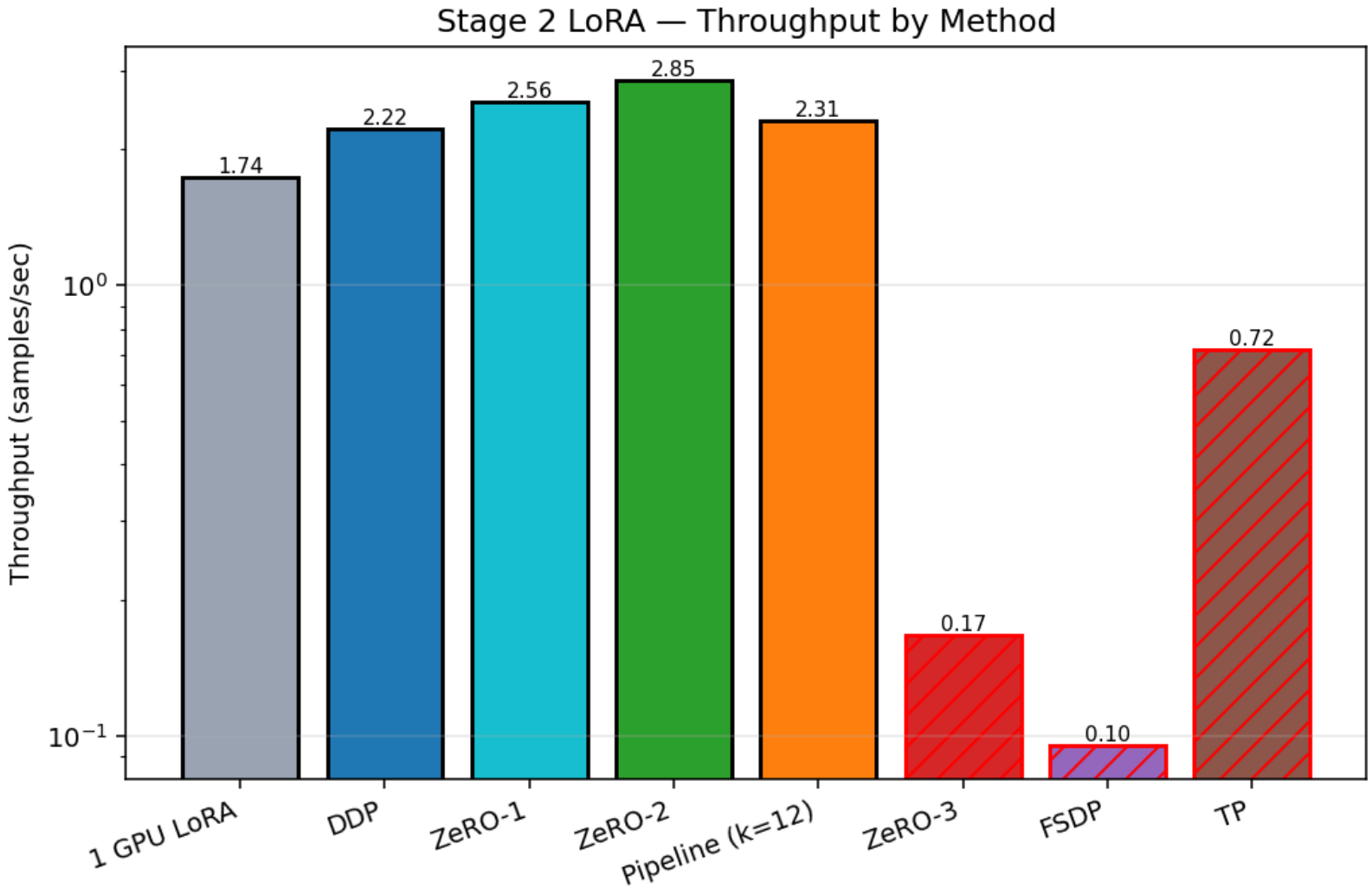
- Ran a full epoch — 1250 steps × effective batch 8 — at 11 s/step.
- Wall time: 3.85 h. Throughput: 0.72 sps — 0.32× DDP, but 4× faster than ZeRO-3 and 8× faster than FSDP.
  - TP ships ~2 GB of activations per microbatch vs ZeRO-3's ~14 GB of weights — order of magnitude less.
- Peak GPU memory: 30.6 GiB. GPU util: 77.7%. Final loss: 1.03 — matches DDP at full epoch.
- Takeaway: TP is the only model-parallel method actually usable on this link.
  - Still 3× slower than DDP — per-layer collectives accumulate Ethernet latency.
  - Designed for NVLink, where AllReduce is essentially free.



Putting it all together

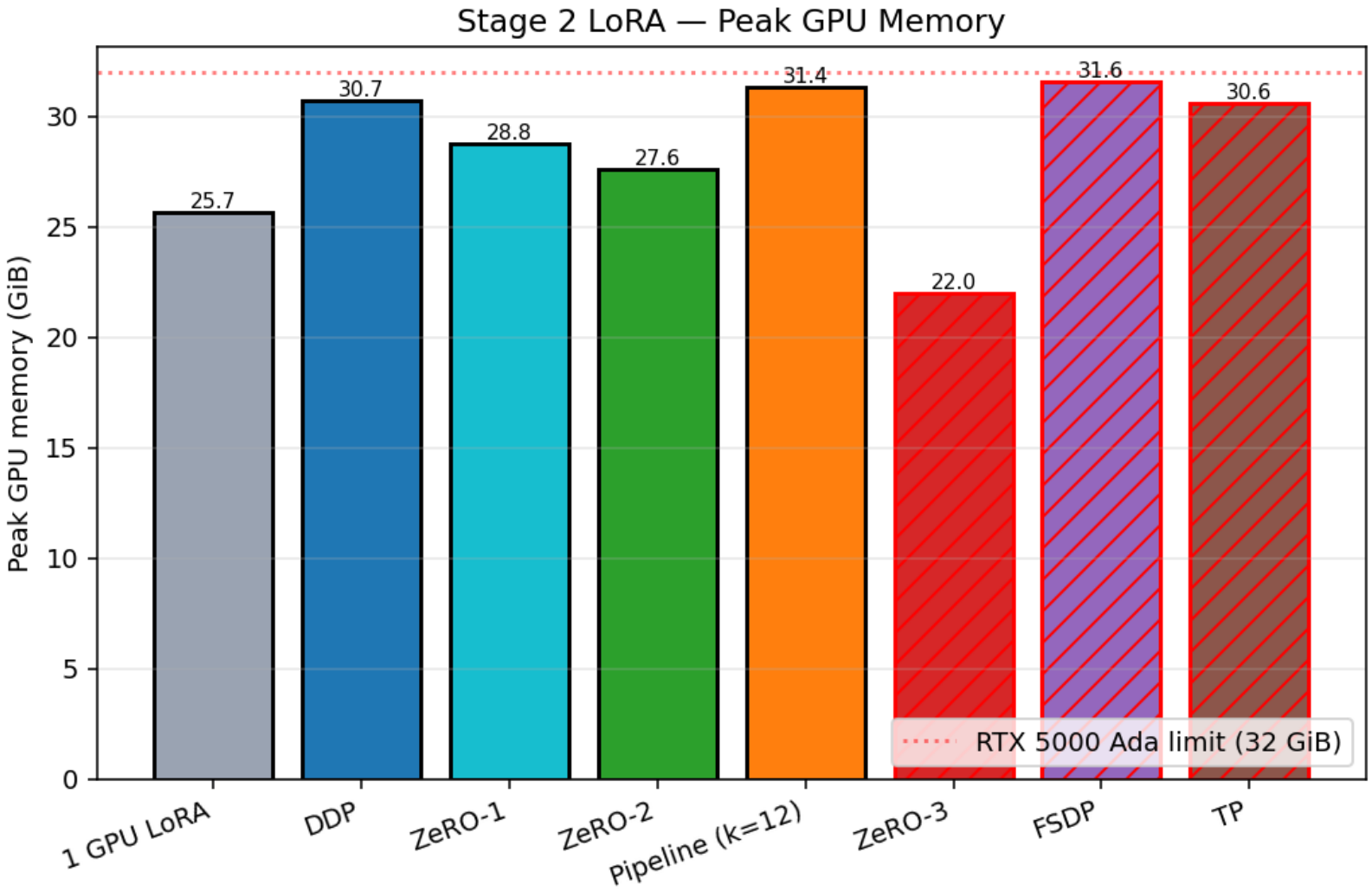
Throughput by method

ZeRO-2 wins, ZeRO-1 close behind. ZeRO-3 / FSDP collapse on Ethernet; TP in the middle.



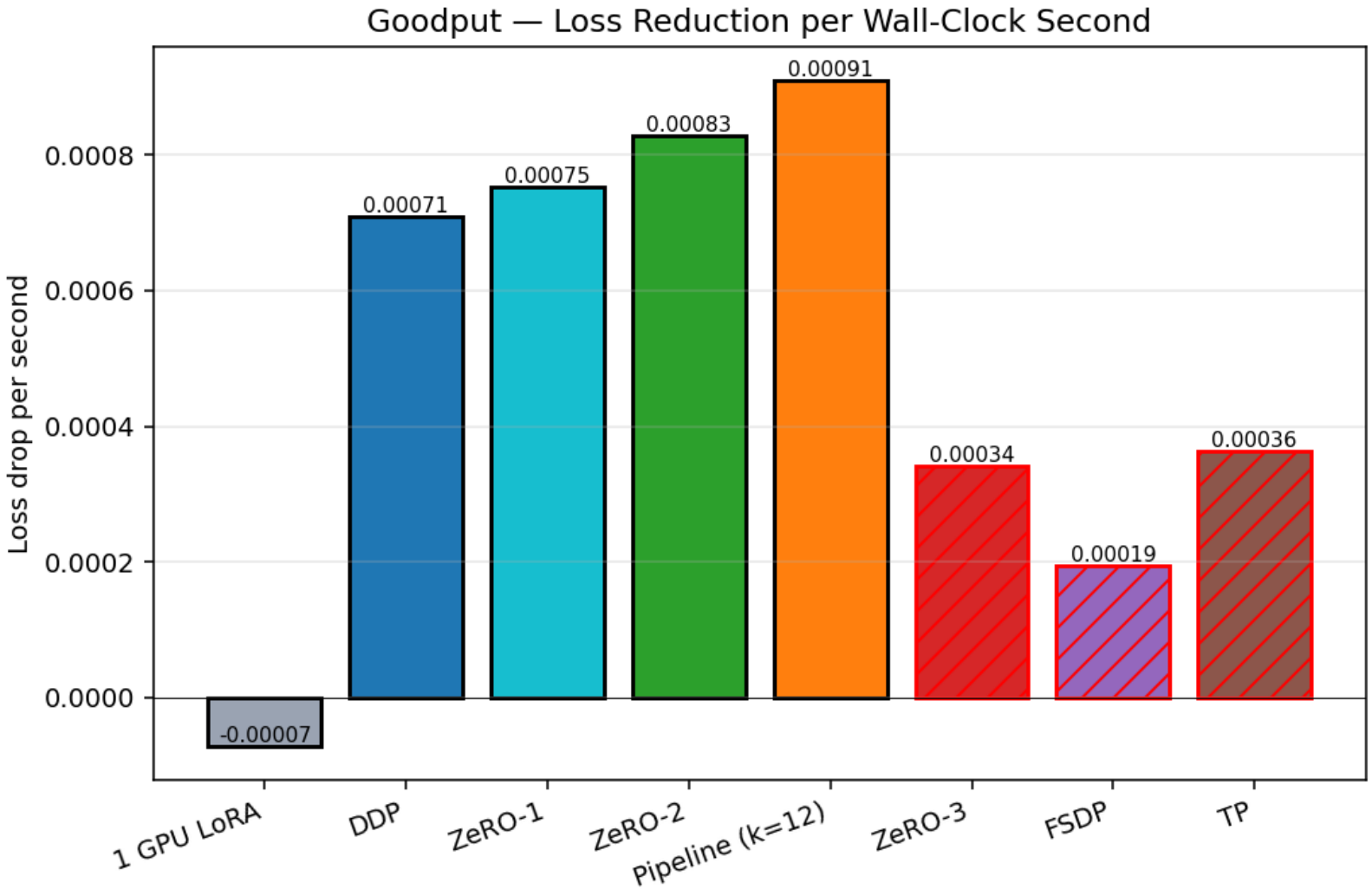
Peak GPU memory

ZeRO-3 shards the most. FSDP and TP saturate the card. ZeRO-2 hits a sweet spot.



Goodput — loss reduction per second

Full-epoch methods cluster on top; cross-layer-comm methods (ZeRO-3 / FSDP / TP) sit below, ordered by throughput.



Loss vs step (smoothed)

All methods follow the same loss-vs-step curve at matched batch. TP's curve extends further because its smaller batch (8 vs 32) needs 1250 steps for a full epoch.

What we learned

- ZeRO-2 wins for LoRA on slow interconnects (1.28× DDP). ZeRO-1 is close behind at 1.15×.
- Pipeline parallelism made the model fit but not faster — Gloo cross-node transfers cost more than DDP's single AllReduce.
- ZeRO-3 and FSDP fall apart over 1 Gbps Ethernet — per-layer AllGather of bf16 weights is intractable.
  - Implementation matters: DeepSpeed ZeRO-3 is ~1.7× faster than PyTorch FSDP, thanks to better overlap.
- Tensor Parallel is the only model-parallel method actually usable here — 3× slower than DDP, but ships ~7× less data than ZeRO-3.
- The heavy methods (ZeRO-3, FSDP, TP) all need NVLink or InfiniBand. Wrong tool for Ethernet.
- A 7B model with LoRA fits comfortably on one 32 GiB card — pick a data-parallel variant.

Thank you

Questions?

github.com/rassulmagauin/ml710 · ZERO2_RESULTS.md · results_explanation.md